STIR vs FRI: Verifier Hash Complexity for ZisK

A per-AIR comparison of the number of Poseidon permutations a verifier performs, for FRI versus STIR (ePrint 2024/390), at 128-bit security.

Includes: the hash-count model, the answer to “is STIR round 0 the same as FRI?” with paper citations, and the measured results.

Headline: across the 35 ZisK base AIRs + the recursive circuit, STIR wins on every AIR; the verifier does ≈1.27× fewer Poseidon hashes overall.

1. What is being counted

The verifier of a FRI/STIR proof spends its time computing Poseidon hashes. We count Poseidon permutations in the two modes the prover system actually uses:

- Merkle-tree COMPRESSION (arity 4): one permutation per level on an authentication path. A path over a 2N domain costs $\lceil N / \log_2(4) \rceil \leq$ lastLevelVerification permutations — one per level, NOT (arity−1) per level.
- Sponge LINEAR hash (rate 12, capacity 4): hashing a leaf of w field elements costs $\lceil w / 12 \rceil$ permutations.

The verifier's work splits into two parts:

- First-phase commitment openings (“check f is built correctly”): per query, open one leaf in each committed-stage Merkle tree (Fixed, Stage1, Stage2, Q) — walk the path and sponge-hash that stage's row width.
- Low-degree-test openings: FRI re-opens its full query set at EVERY fold layer; STIR opens t_i leaves at each round, where t_i shrinks as the rate improves.

Source of the model (verified against the C++ verifier)

pil2-stark/src/starkpil/merkleTree/merkleTreeGL.cpp — calculateRootFromProof

One Poseidon permuteTrunc per tree level. getMerkleProofLength() = $\lceil \log_2(\text{height}) / \log_2(\text{arity}) \rceil$ − lastLevelVerification. The (arity−1) factor appears only in getMerkleProofSize (the proof byte size / sibling count), never in the hash count.

pil2-stark/src/goldilocks/.../poseidon_goldilocks.hpp

Arity-4 Merkle Poseidon has SPONGE_WIDTH = 16, CAPACITY = 4, RATE = 12. linear_hash absorbs RATE = 12 elements per permutation.

2. Is STIR round 0 the same as FRI?

Short answer: YES, when both test the initial Reed–Solomon instance to the same proximity parameter δ. Round 0 is the plain RS proximity test of the input; STIR adds nothing to it. STIR's entire advantage lives in rounds i ≥ 1, where it improves the rate.

Why — with paper citations (ePrint 2024/390)

Theorem 5.1, p.22 — two SEPARATE query complexities

"Query complexity to the input: $\lambda / -\log(1-\delta)$." "Query complexity to the proof strings: $O_k(\log d + \lambda \cdot \log(\log d / \log(1/\rho)))$." The input (round 0) has its own formula, distinct from the folding rounds.

Construction 5.2, Verifier main loop item 1(a), p.23

Round 1 queries $\text{Fold}(f_0, k_0, r_0)$ at t_0 points — i.e. it queries the INPUT function f_0 directly. The text notes: "If i = 1 this is where things end" — round 0 is the simple proximity test; later rounds query virtual folded functions.

Appendix C.1 / C.2, 'Input query complexity', p.53 & p.55

$t_0 \le \lambda / -\log(1-\delta')$, where $\delta' = \min\{\delta, 1 - 1.05\cdot\sqrt{\rho}\}$ (provable) or $\min\{\delta, 1 - 1.05\cdot\rho\}$ (conjectured). Driven by the proximity parameter — the same quantity that governs FRI's first-layer query count.

Section 1.1, 'Comparison to prior works', p.4

"the main difference between the two is that FRI does not enjoy a decrease in rate between rounds, which requires more queries." The difference is BETWEEN rounds, not at the input.

Appendix C, Fact C.2, p.51 — where STIR's win comes from

$\rho_i = (2/k)^i \cdot \rho$. The rate improves geometrically each round, so t_i ($i \ge 1$) shrinks. This does not touch t_0.

Takeaway: round 0 is the entry toll set by the starting rate, paid identically by FRI and STIR. FRI re-pays that toll at every fold layer; STIR pays a geometrically shrinking cost.

3. Round-0 example (Main AIR, rate 1/2)

With protocol security 106 bits (= 128 target − 22 grinding PoW bits), at rate 1/2 (log_inv_rate = 1):

```
STIR round 0 :  t_0 = ceil(2*106 / 1)  = 212 queries
FRI 1st layer:  ~218 queries           (same RS instance, same rate)


STIR per-round (k=4, rate improves each round):
   round 0: lir=1   t=212     <- same as FRI
   round 2: lir=2   t=106
   round 2: lir=3   t=71
   ...      ...      (geometric shrink)


FRI: ~218 queries at EVERY one of its fold layers (rate never improves).
```

So round 0 being equal is exactly why STIR cannot beat FRI by a huge margin: the first, most expensive commitment is at the un-improved rate for both. STIR wins because every subsequent round gets cheaper, while FRI keeps re-paying the round-0 price.

4. Measured results: verifier Poseidon hashes per AIR

128-bit target, Merkle arity 4, sponge rate 12, lastLevelVerification = 2, grinding 22 bits. STIR folds to stopping degree 2^6. Run locally via:

```
cd pil2-proofman/setup

cargo test -p pil2-stark-setup zisk_air_hash_table -- --ignored --nocapture
```

```
ZisK verifier Poseidon hashes @ 128 bits, MT arity 4, sponge rate 12
AIR                    nBits  rate |   FRI_q  FRI_hash |    k   M STIR_hash |  STIR_q   ratio | winner
-------------------------------------------------------------------------------------------------------
Main                      22  2^-1 |     218     21800 |    4   8     16035 |     603    1.36 |   STIR
Rom                       22  2^-1 |     217     20832 |    4   8     15187 |     603    1.37 |   STIR
Mem                       22  2^-1 |     217     21049 |    4   8     15399 |     603    1.37 |   STIR
InputData                 21  2^-1 |     217     18445 |    4   8     14170 |     603    1.30 |   STIR
RomData                   21  2^-1 |     216     18144 |    4   8     13958 |     603    1.30 |   STIR
MemAlign                  21  2^-1 |     217     18879 |    4   8     14594 |     603    1.29 |   STIR
MemAlignByte              22  2^-1 |     217     21049 |    4   8     15399 |     603    1.37 |   STIR
MemAlignReadByte          22  2^-1 |     217     20832 |    4   8     15187 |     603    1.37 |   STIR
MemAlignWriteByte         22  2^-1 |     217     21049 |    4   8     15399 |     603    1.37 |   STIR
Arith                     21  2^-1 |     217     19530 |    4   8     15230 |     603    1.28 |   STIR
Binary                    22  2^-1 |     218     21800 |    4   8     16035 |     603    1.36 |   STIR
BinaryAdd                 22  2^-1 |     217     20832 |    4   8     15187 |     603    1.37 |   STIR
BinaryExtension           22  2^-1 |     218     21582 |    4   8     15823 |     603    1.36 |   STIR
Add256                    20  2^-1 |     217     19313 |    4   7     15076 |     579    1.28 |   STIR
ArithEq                   20  2^-1 |     219     19272 |    4   7     14864 |     579    1.30 |   STIR
ArithEq384                20  2^-1 |     219     19053 |    4   7     14652 |     579    1.30 |   STIR
Keccakf                   17  2^-1 |     219     67890 |    4   6     64037 |     552    1.06 |   STIR
Sha256f                   18  2^-1 |     218     17004 |    4   6     13938 |     552    1.22 |   STIR
Poseidon                  17  2^-1 |     217     16492 |    4   6     14429 |     552    1.14 |   STIR
Blake2br                  18  2^-1 |     218     18530 |    4   6     15422 |     552    1.20 |   STIR
Dma                       21  2^-1 |     217     18879 |    4   8     14594 |     603    1.29 |   STIR
DmaMemCpy                 21  2^-1 |     217     18662 |    4   8     14382 |     603    1.30 |   STIR
DmaInputCpy               21  2^-1 |     217     18662 |    4   8     14382 |     603    1.30 |   STIR
Dma64Aligned              21  2^-1 |     217     19096 |    4   8     14806 |     603    1.29 |   STIR
Dma64AlignedInputCpy      21  2^-1 |     217     19096 |    4   8     14806 |     603    1.29 |   STIR
Dma64AlignedMemSet        21  2^-1 |     217     18662 |    4   8     14382 |     603    1.30 |   STIR
Dma64AlignedMem           21  2^-1 |     217     18879 |    4   8     14594 |     603    1.29 |   STIR
Dma64AlignedMemCpy        21  2^-1 |     217     19096 |    4   8     14806 |     603    1.29 |   STIR
DmaUnaligned              21  2^-1 |     217     18445 |    4   8     14170 |     603    1.30 |   STIR
DmaPrePost                21  2^-1 |     218     19838 |    4   8     15442 |     603    1.28 |   STIR
DmaPrePostMemCpy          21  2^-1 |     217     19530 |    4   8     15230 |     603    1.28 |   STIR
DmaPrePostInputCpy        21  2^-1 |     217     18879 |    4   8     14594 |     603    1.29 |   STIR
VirtualTableZisk0         21  2^-1 |     218     20492 |    4   8     16078 |     603    1.27 |   STIR
VirtualTableZisk1         21  2^-1 |     218     19838 |    4   8     15442 |     603    1.28 |   STIR
Recursive                 17  2^-3 |      72      5904 |    4   6      5635 |     285    1.05 |   STIR
-------------------------------------------------------------------------------------------------------
TOTAL  FRI_hash=717335  STIR_hash=563364  overall_ratio=1.27x
```

How to read a row

- FRI_q / FRI_hash: FRI query count (per layer) and total verifier permutations.
- k / M: STIR's chosen fold factor and number of folding rounds.
- STIR_hash / STIR_q: STIR total permutations and summed-over-rounds query count.
- ratio = FRI_hash / STIR_hash (higher = STIR saves more).

Notes: STIR wins on every AIR. The narrowest win is Keccakf (≈1.06×) — its huge Stage1 width (2137 columns) makes the first-phase commitment sponge dominate, a cost shared by both protocols. The recursive circuit (rate 2^{-3}) has few hashes overall because the low rate already makes each query strong.

5. Caveats & honesty notes

- Absolute counts depend on the real FRI step schedule and lastLevelVerification per AIR. The table uses fold-by-4 down to 2^6 and llv=2; the FRI/STIR RATIO is robust to these, the absolute numbers are not.
- STIR repetitions use the conjectured-soundness formula (c_1=c_2=c_3=1), matching the paper's benchmark configuration (Section 6.2).
- Grinding is modeled as a single global 22-bit proof-of-work (paper Section 6.2), reducing the IOP soundness requirement to 2^{-106}.
- Round 0 uses the provable bound $t_0 = 2 \cdot \lambda\_prot / log\_inv\_rate$, which matches FRI. At the conjectured input bound ($\delta' = 1 - 1.05 \cdot \rho$, p.55) round 0 could be cheaper, but FRI can use the same bound, so they stay matched.

All figures are produced by the calculator in setup/pil2-stark/src/types/security.rs and are reproducible with the cargo command in Section 4.